



Université de Tunis El Manar
École Nationale d'Ingénieurs de
Tunis



Rapport final sujet numéro 4

Calcul de la valeur efficace d'un signal sinusoïdal injecté sur canal 1

Réalisé par :

Maram TURKI

Hamza MAMI

2^{ème} année Génie Électrique (GE2)

Encadrés par :

Mr. Jelassi Khaled

Année universitaire : 2025–2026

Table des matières

1	Introduction Générale	3
2	Présentation du matériel et des outils	4
2.1	Microcontrôleur STM32F4	4
2.2	Outils logiciels	4
2.3	Présentation pratique	5
2.4	Récapitulatif du matériel et des outils	5
3	Étude théorique	6
3.1	Signal sinusoïdal	6
3.2	Valeur efficace (RMS)	6
3.2.1	Formule générale	6
3.2.2	Méthode discrète utilisée	7
3.2.3	Cas particulier : sinus pur	7
3.3	Applications	7
4	Principe de fonctionnement global	8
4.1	Vue d'ensemble du système	8
4.2	Schéma bloc du système	8
4.3	Rôle du DAC	9
4.4	Rôle de l'ADC	9
4.5	Rôle du microcontrôleur	9
4.6	Gestion temporelle par interruption SysTick	10
4.7	Échantillonnage et réglage de la fréquence par diviseur	10
4.7.1	Fréquence d'échantillonnage	11
4.7.2	Table sinus et fréquence du signal généré	11
4.7.3	Réglage de la fréquence par potentiomètre	11
4.7.4	Respect du théorème de Nyquist	11
4.8	Calcul de la valeur efficace	12
5	Description logicielle	14
5.1	main.c	14
5.2	adc.c	14
5.3	dac.c	15
5.4	config.c	15
5.5	SysTick_Handler (interruption)	16
6	Résultats expérimentaux	17
6.1	Observation du signal	17
6.2	Variation de la fréquence	18
7	Conclusion	19
A	Annexes	20
A.1	Code source complet	20
B	Montage complet du système	23

Table des figures

1	Carte STM32F407	4
2	Logo Keil uVision	4
3	Signal sinusoïdal	6
4	Illustration de la valeur efficace d'un signal sinus	7
5	Schéma bloc du projet	8
6	Principe de DAC	9
7	Principe de l'ADC	9
8	Principe de l'échantillonnage	10
9	Chaine d'opérations à chaque interruption	13
10	Signal sinusoïdal généré et visualisé sur l'oscilloscope	17
11	Capture du signal sur le Logic Analyzer	17
12	Signal sinusoïdal à 39 Hz	18
13	Signal sinusoïdal à 78 Hz	18
14	Montage complet du système	23

1 Introduction Générale

La mesure de la valeur efficace d'un signal électrique est une opération fondamentale en électronique et en électrotechnique, notamment dans les domaines de l'instrumentation, de l'automatisation industrielle et des systèmes de mesure. Contrairement à la valeur instantanée ou à la valeur moyenne, la valeur efficace permet de caractériser l'énergie réellement fournie par un signal alternatif et constitue une grandeur essentielle pour l'analyse et le dimensionnement des systèmes électriques.

Dans ce projet, nous nous intéressons au calcul de la valeur efficace d'un signal sinusoïdal à l'aide d'un microcontrôleur de la famille STM32F4. Le signal est d'abord généré numériquement puis converti en signal analogique grâce au convertisseur numérique-analogique (DAC) intégré au microcontrôleur. Ce signal analogique est ensuite réinjecté vers le convertisseur analogique-numérique (ADC) afin d'être acquis et traité numériquement par le microcontrôleur.

L'objectif principal de ce travail est de mettre en œuvre une chaîne complète de traitement du signal, allant de la génération analogique jusqu'au calcul de la valeur efficace. La méthode retenue repose sur la détermination de la valeur maximale du signal sinusoïdal mesuré, suivie de l'application de la relation théorique reliant la valeur maximale à la valeur efficace, à savoir une division par racine de 2. La fréquence et l'amplitude du signal sont rendues variables à l'aide de potentiomètres, permettant ainsi d'étudier l'influence de ces paramètres sur les résultats obtenus.

Ce projet permet ainsi de consolider les notions liées aux convertisseurs ADC et DAC, à la programmation temps réel à l'aide des interruptions, ainsi qu'à l'analyse et au traitement des signaux analogiques à l'aide d'un microcontrôleur.

2 Présentation du matériel et des outils

2.1 Microcontrôleur STM32F4

Le projet utilise le microcontrôleur **STM32F407VGT6**, basé sur l'architecture **ARM Cortex-M4** 32 bits, cadencé jusqu'à 168 MHz. Ce microcontrôleur possède de la mémoire Flash et RAM suffisantes pour stocker le code et les buffers nécessaires au traitement du signal.

Les périphériques utilisés sont :

- **DAC (Digital to Analog Converter)** : résolution 12 bits, utilisé pour générer un signal sinusoïdal sur la sortie PA4.
- **ADC (Analog to Digital Converter)** : résolution 12 bits, utilisé pour mesurer le signal en retour du DAC afin de calculer la valeur efficace (RMS).

L'intégration de ces périphériques sur la puce permet un traitement du signal rapide et précis sans composants externes supplémentaires.

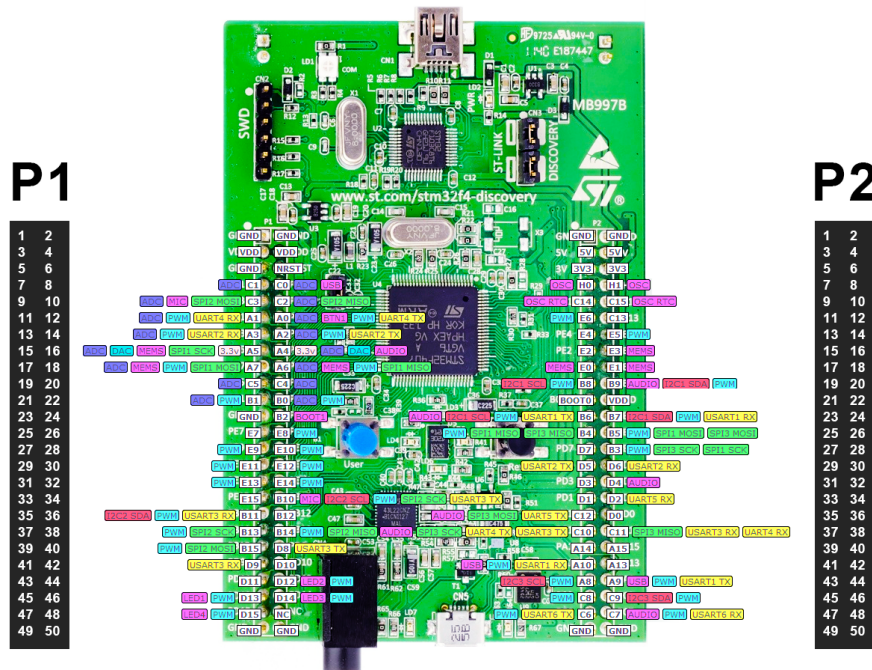


FIGURE 1 – Carte STM32F407

2.2 Outils logiciels

Les outils logiciels utilisés pour ce projet sont les suivants :

- **Keil μ Vision 5** : IDE permettant l'écriture du code, sa compilation et le débogage sur le microcontrôleur.



FIGURE 2 – Logo Keil uVision

- **STM32 Standard Peripheral Library** (STM32F4xx_StdPeriph_Driver) : fournit les fichiers et fonctions pour configurer facilement les périphériques comme le DAC, l'ADC et les GPIO.
- **Oscilloscope** : permet de visualiser le signal sinusoïdal généré par le DAC et de vérifier sa forme et sa fréquence.
- **Logic Analyzer** : utilisé pour vérifier les échantillons numériques du signal et le timing.

2.3 Présentation pratique

Le câblage et l'utilisation des composants sont organisés comme suit :

- PA4 : sortie DAC et entrée ADC (bouclage pour mesure du signal).
- PC1 : potentiomètre pour ajuster l'amplitude du signal.
- PC2 : potentiomètre pour ajuster la fréquence du signal.

Le flux du signal peut être résumé ainsi : **DAC** → **signal analogique** → **ADC** → **traitement RMS** → **affichage/debug**.

2.4 Récapitulatif du matériel et des outils

Matériel / Logiciel	Description / Rôle
STM32F407VGT6	Microcontrôleur ARM Cortex-M4 avec DAC et ADC intégrés
Keil µVision 5	IDE pour écrire, compiler et déboguer le code
STM32 Standard Peripheral Library	Librairie pour configurer DAC, ADC, GPIO et SysTick
Oscilloscope	Visualisation du signal sinusoïdal généré
Logic Analyzer	Vérification numérique des échantillons et du timing
Potentiomètres PC1/PC2	Ajustement de l'amplitude et de la fréquence du signal

TABLE 1 – Récapitulatif du matériel et des outils utilisés.

3 Étude théorique

3.1 Signal sinusoïdal

Un signal sinusoïdal est une variation périodique continue pouvant être décrite par l'expression :

$$v(t) = V_{\max} \cdot \sin(2\pi ft + \phi)$$

où :

- $v(t)$: tension instantanée (en volts),
- $V_{\max}$: amplitude maximale du signal,
- f : fréquence du signal (Hz),
- ϕ : phase initiale (radians).

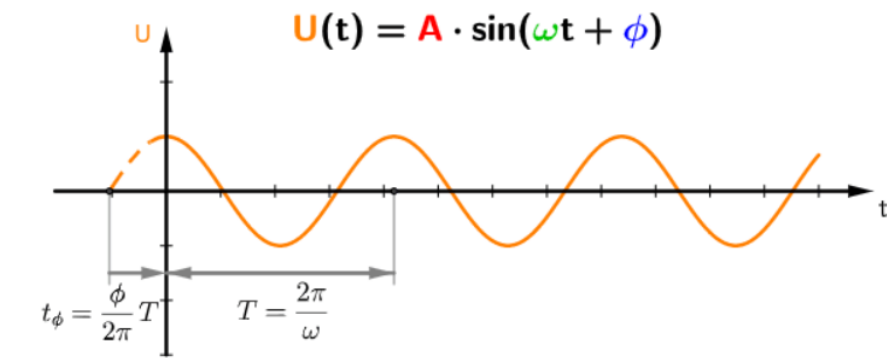


FIGURE 3 – Signal sinusoïdal

Paramètres du signal :

- **Amplitude** : valeur maximale du signal.
- **Fréquence** : nombre de cycles par seconde.
- **Phase** : décalage initial dans le temps.

Dans ce projet, le signal est généré par le DAC du STM32F4, centré sur 0 V, avec amplitude et fréquence ajustables via des potentiomètres.

3.2 Valeur efficace (RMS)

La **valeur efficace (RMS, Root Mean Square)** permet de caractériser l'énergie transportée par un signal alternatif et correspond à la valeur continue qui produirait la même puissance sur une résistance.

3.2.1 Formule générale

Pour un signal $v(t)$ sur une période T :

$$V_{\text{eff}} = \sqrt{\frac{1}{T} \int_0^T v^2(t) dt}$$

3.2.2 Méthode discrète utilisée

Pour un signal échantillonné $v[n]$, la valeur RMS se calcule par :

$$V_{\text{eff}} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} v[n]^2}$$

où :

- N : nombre d'échantillons par période,
- $v[n]$: tension instantanée après suppression de l'offset DC.

Dans notre projet, le signal DAC est centré sur $V_{\text{REF}}/2$ (1.65 V), donc l'offset est supprimé avant le calcul RMS :

$$v_{\text{ac}}[n] = v[n] - \frac{V_{\text{REF}}}{2}$$

Ensuite, la valeur efficace est calculée par :

$$V_{\text{eff}} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} v_{\text{ac}}^2[n]}$$

Cette méthode permet d'obtenir un RMS précis même si la forme du signal n'est pas parfaitement sinusoïdale ou si la fréquence varie.

3.2.3 Cas particulier : sinus pur

Pour un sinus pur $v(t) = V_{\text{max}} \sin(2\pi ft)$, la valeur RMS est proche de :

$$V_{\text{eff}} \approx \frac{V_{\text{max}}}{\sqrt{2}} \approx 0.707 \cdot V_{\text{max}}$$

L'avantage de la méthode discrète est qu'elle peut être utilisée pour tous les types de signaux mesurés numériquement.

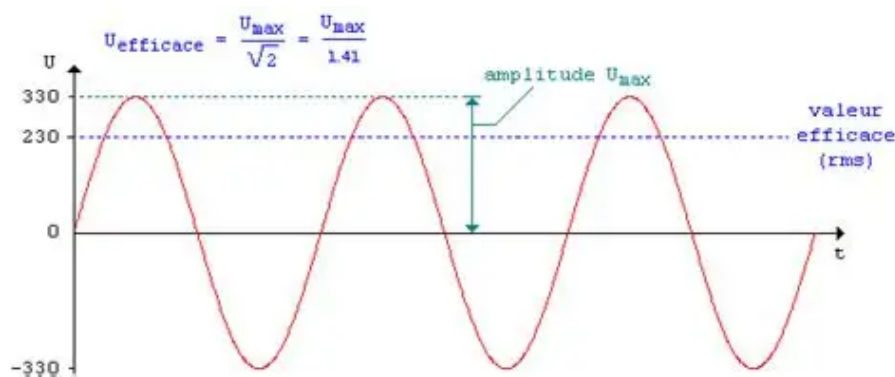


FIGURE 4 – Illustration de la valeur efficace d'un signal sinus

3.3 Applications

La valeur RMS est utilisée pour :

- Dimensionner des charges résistives et électroniques,
- Comparer la puissance de signaux alternatifs à celle d'une source continue,
- Vérifier la précision d'un signal généré par un DAC.

4 Principe de fonctionnement global

4.1 Vue d'ensemble du système

Le système réalisé permet la génération d'un signal sinusoïdal analogique à l'aide du DAC interne du microcontrôleur STM32F407. L'amplitude et la fréquence de ce signal sont réglables par l'utilisateur à l'aide de deux potentiomètres. Le signal généré est ensuite réinjecté vers un canal ADC afin d'en calculer la valeur efficace (RMS) par traitement numérique.

L'ensemble du fonctionnement est synchronisé par une interruption périodique *SysTick* configurée à une fréquence de 20 kHz, assurant à la fois la génération du signal et son acquisition.

4.2 Schéma bloc du système

Le système peut être décrit par le schéma fonctionnel suivant :

- Deux potentiomètres fournissent les consignes de fréquence et d'amplitude.
- L'ADC convertit ces signaux analogiques en valeurs numériques.
- Le microcontrôleur calcule les échantillons du signal sinusoïdal.
- Le DAC génère le signal analogique correspondant.
- Le signal est réinjecté vers l'ADC pour le calcul RMS.

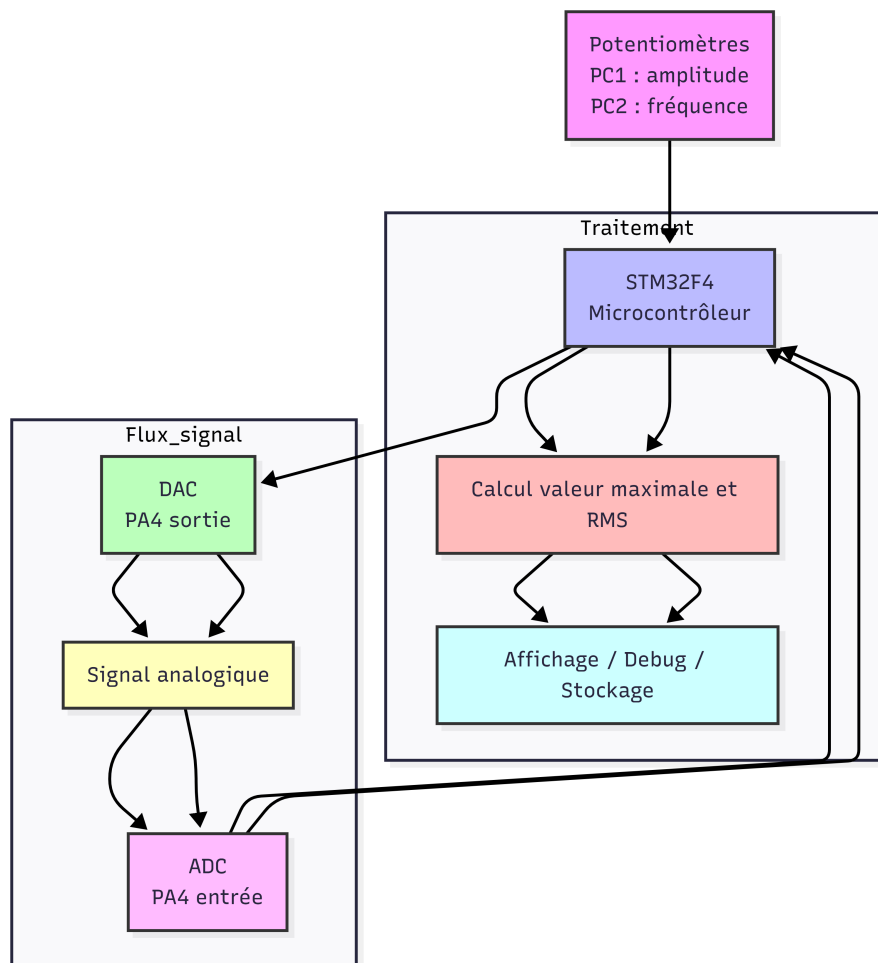


FIGURE 5 – Schéma bloc du projet

4.3 Rôle du DAC

Le DAC (Digital to Analog Converter) interne du STM32F407 est chargé de convertir les valeurs numériques calculées par le microcontrôleur en un signal analogique sinusoïdal.

Une table de sinus discrétisée de $N = 256$ points est préalablement générée en mémoire. À chaque interruption SysTick, un échantillon de cette table est sélectionné, normalisé dans l'intervalle $[-1; +1]$, multiplié par le facteur d'amplitude et décalé par un offset continu de 1,65 V afin de respecter la plage de sortie du DAC (0 à 3,3 V).

Le DAC fournit ainsi un signal analogique réel, exploitable par le reste du système.



FIGURE 6 – Principe de DAC

4.4 Rôle de l'ADC

L'ADC (Analog to Digital Converter) assure deux fonctions principales :

- La lecture des potentiomètres de fréquence et d'amplitude.
- L'acquisition du signal sinusoïdal généré par le DAC.

Le signal analogique issu du DAC est converti en une valeur numérique sur 12 bits selon la relation :

$$v_{\text{adc}} = \frac{\text{ADC} \times V_{\text{ref}}}{4095}$$

Un offset continu correspondant à la moitié de la tension de référence est ensuite supprimé afin d'obtenir uniquement la composante alternative du signal :

$$v_{\text{ac}} = v_{\text{adc}} - \frac{V_{\text{ref}}}{2}$$

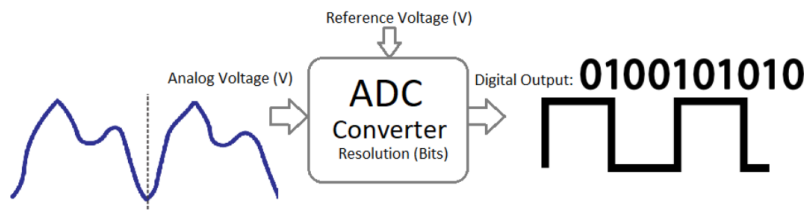


FIGURE 7 – Principe de l'ADC

4.5 Rôle du microcontrôleur

Le microcontrôleur STM32F407 constitue le cœur du système et assure les fonctions suivantes :

- Génération temporelle grâce à l'interruption SysTick.

- Gestion de la fréquence du signal par ajustement du diviseur de fréquence.
- Calcul des valeurs numériques du sinus à partir d'une table pré-calculée.
- Acquisition des échantillons du signal analogique via l'ADC.
- Calcul de la valeur efficace du signal.

4.6 Gestion temporelle par interruption SysTick

Le fonctionnement du système repose sur une interruption périodique générée par le timer **SysTick** intégré au cœur *Cortex-M4* du microcontrôleur STM32F4. Cette interruption assure la synchronisation temporelle de l'ensemble des tâches de génération, d'acquisition et de traitement du signal.

La fréquence de l'interruption SysTick est fixée à 20 kHz, ce qui correspond à une période d'échantillonnage de 50 μ s. Ce choix garantit un pas de temps constant, indispensable pour la génération d'un signal sinusoïdal stable ainsi que pour un calcul précis de la valeur efficace.

À chaque déclenchement de l'interruption, les opérations suivantes sont exécutées de manière déterministe :

- lecture des potentiomètres permettant de régler l'amplitude et la fréquence du signal sinusoïdal ;
- mise à jour de l'indice de la table sinus et calcul de la valeur numérique à appliquer au convertisseur numérique-analogique (DAC) ;
- génération du signal analogique sinusoïdal en sortie du DAC ;
- acquisition du signal analogique via le convertisseur analogique-numérique (ADC) ;
- conversion de la valeur ADC en tension et suppression de l'offset continu ;
- accumulation de la somme des carrés des échantillons en vue du calcul RMS.

Après l'acquisition d'un nombre déterminé d'échantillons, la valeur efficace est calculée selon la méthode de la racine de la moyenne des carrés. L'utilisation de l'interruption SysTick permet d'assurer un échantillonnage uniforme du signal, condition essentielle pour la validité mathématique du calcul RMS.

4.7 Échantillonnage et réglage de la fréquence par diviseur

Dans ce projet, la génération et le traitement du signal sont entièrement réalisés de manière numérique à l'aide du microcontrôleur STM32F407. Le fonctionnement du système repose sur une interruption périodique fournie par le timer **SysTick**, qui impose la fréquence d'échantillonnage du système.

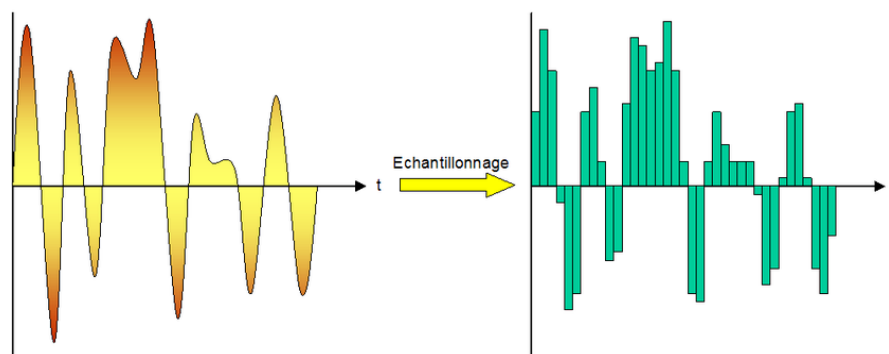


FIGURE 8 – Principe de l'échantillonnage

4.7.1 Fréquence d'échantillonnage

L'interruption SysTick est configurée à une fréquence fixe :

$$F_s = 20 \text{ kHz}$$

Cela signifie que le programme exécute l'algorithme de génération et de traitement du signal **20 000 fois par seconde**. À chaque interruption, les opérations suivantes sont réalisées :

- Calcul de la nouvelle valeur du sinus et envoi au DAC
- Acquisition du signal par l'ADC
- Mise à jour du calcul de la valeur efficace (RMS)

Cette fréquence définit le pas temporel de l'échantillonnage du signal.

4.7.2 Table sinus et fréquence du signal généré

Le signal sinusoïdal est stocké sous forme d'une table de $N = 256$ échantillons représentant une période complète. Si chaque échantillon était envoyé au DAC à chaque interruption, la fréquence du signal serait :

$$f = \frac{F_s}{N} = \frac{20\,000}{256} \approx 78 \text{ Hz}$$

Pour permettre la variation de la fréquence, un **diviseur de fréquence logiciel** est utilisé :

$$f = \frac{F_s}{N \times \text{diviseur_freq}}$$

Le diviseur est calculé à partir du potentiomètre de fréquence :

$$\text{diviseur_freq} = \frac{F_s}{f_{\text{souhaite}} \times N}$$

Ainsi, le microcontrôleur n'avance dans la table sinus qu'une fois toutes les `diviseur_freq` interruptions.

4.7.3 Réglage de la fréquence par potentiomètre

Le potentiomètre connecté sur l'entrée **PC2** est lu par l'ADC et permet de faire varier la fréquence du signal entre 1 Hz et 100 Hz :

```
1 freq = 1.0f + ((float)pot_freq / 4095.0f) * 99.0f;  
2 diviseur_freq = FS / (freq * NPOINTS);
```

Cela permet à l'utilisateur de modifier dynamiquement la fréquence du signal généré.

4.7.4 Respect du théorème de Nyquist

Pour garantir une bonne reconstruction du signal et un calcul RMS précis, la fréquence d'échantillonnage doit vérifier :

$$F_s > 2 \times f_{\text{max}}$$

Dans ce projet :

$$20 \text{ kHz} \gg 2 \times 100 \text{ Hz}$$

Le théorème de Nyquist est donc largement respecté, assurant une forme de sinus fidèle et une mesure RMS précise.

4.8 Calcul de la valeur efficace

Le calcul de la valeur efficace est réalisé par la méthode numérique de la racine de la moyenne des carrés. Pour chaque échantillon acquis, le carré de la valeur instantanée est accumulé :

$$s = \sum_{i=1}^N v_i^2$$

Après N échantillons, la valeur efficace est calculée par :

$$V_{\text{eff}} = \sqrt{\frac{1}{N} \sum_{i=1}^N v_i^2}$$

Cette méthode est indépendante de la forme du signal et fournit une mesure précise de la valeur RMS.

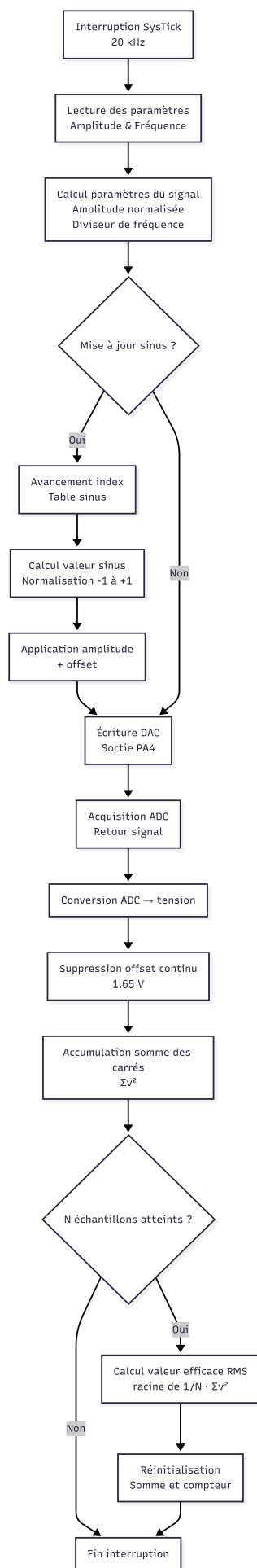


FIGURE 9 – Chaîne d'opérations à chaque interruption

5 Description logicielle

Le projet est structuré de manière modulaire afin de séparer clairement les fonctionnalités : génération du signal, acquisition, configuration et gestion des périphériques. Chaque fichier source est dédié à un rôle spécifique, ce qui facilite la maintenance et la compréhension du code.

5.1 main.c

Rôle : point d'entrée du programme, initialisation des périphériques, activation de l'interruption SysTick et lancement du traitement temps réel.

Listing 1 – main.c – Initialisation du système et boucle principale

```
1 int main(void)
2 {
3     config();          // Configuration horloges GPIO, ADC, DAC
4     dac_init();        // Initialisation DAC
5     adc_init();        // Initialisation ADC
6     init_sin_table();  // Initialisation table sinus
7
8     SysTick_Config(SystemCoreClock / FS); // Activation interruption
        SysTick
9
10    while (1)
11    {
12        // Boucle vide : tout le traitement temps r el
13        // est g r dans SysTick_Handler
14    }
15 }
```

5.2 adc.c

Rôle : configuration du convertisseur analogique-numérique (ADC) et lecture des canaux analogiques (potentiomètres et retour DAC).

Listing 2 – adc.c – Initialisation de l'ADC

```
1 void adc_init(void)
2 {
3     ADC_InitTypeDef ADC_InitStruct;
4     ADC_CommonInitTypeDef ADC_CommonInitStruct;
5
6     // Configuration ADC1 : 12 bits, conversion unique, d clenchement
        logiciel
7     ADC_InitStruct.ADC_Resolution = ADC_Resolution_12b;
8     ADC_InitStruct.ADC_ContinuousConvMode = DISABLE;
9     ADC_InitStruct.ADC_DataAlign = ADC_DataAlign_Right;
10    ADC_InitStruct.ADC_NbrOfConversion = 1;
11    ADC_Init(ADC1, &ADC_InitStruct);
12    ADC_Cmd(ADC1, ENABLE);
13 }
14
15 // Lecture d un canal ADC
```

```

16 uint16_t readADC1(uint8_t channel)
17 {
18     ADC-RegularChannelConfig(ADC1, channel, 1, ADC_SampleTime_3Cycles);
19     ADC_SoftwareStartConv(ADC1);
20     while (!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));
21     return ADC_GetConversionValue(ADC1);
22 }

```

Explication : L'ADC permet d'acquérir les signaux analogiques de manière précise. Les potentiomètres pilotent l'amplitude et la fréquence du signal, tandis que l'ADC du retour DAC est utilisé pour calculer la valeur efficace RMS.

5.3 dac.c

Rôle : configuration du convertisseur numérique-analogique (DAC) pour générer un signal sinusoïdal variable en amplitude et fréquence.

Listing 3 – dac.c – Initialisation du DAC et sortie analogique

```

1 void dac_init(void)
2 {
3     DAC_InitTypeDef DAC_InitStruct;
4
5     DAC_InitStruct.DAC_Trigger = DAC_Trigger_None; // mise jour
6     DAC_InitStruct.DAC_OutputBuffer = DAC_OutputBuffer_Enable;
7     DAC_Init(DAC_Channel_1, &DAC_InitStruct);
8
9     DAC_Cmd(DAC_Channel_1, ENABLE); // Activation DAC
10 }

```

Explication : Le DAC est piloté dans la routine SysTick pour générer le signal sinusoïdal en sortie PA4 avec un pas de temps constant.

5.4 config.c

Rôle : activation des horloges pour tous les périphériques utilisés (GPIO, ADC, DAC).

Listing 4 – config.c – Activation des horloges

```

1 void config(void)
2 {
3     // Horloges GPIO
4     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA | RCC_AHB1Periph_GPIOC,
5                             ENABLE);
6
7     // Horloges DAC et ADC
8     RCC_APB1PeriphClockCmd(RCC_APB1Periph_DAC, ENABLE);
9     RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);
10 }

```

Explication : Centraliser les horloges dans un fichier dédié permet une initialisation claire et évite les conflits ou doublons.

5.5 SysTick_Handler (interruption)

Rôle : routine d'interruption exécutée à chaque déclenchement de SysTick (20 kHz). Elle réalise la génération du signal, l'acquisition ADC et le calcul RMS.

Listing 5 – Routine SysTick – Génération du signal et calcul RMS

```
1 void SysTick_Handler(void)
2 {
3     // Lecture potentiom tres
4     pot_amp = readADC1(11);
5     pot_freq = readADC1(12);
6
7     // Calcul amplitude et fr quence
8     amplitude = (float)pot_amp / 4095.0f;
9     freq = 1.0f + ((float)pot_freq / 4095.0f) * 99.0f;
10
11    // G n ration signal sinuso dal
12    float sin_norm = ((float)sin_table[index_sin] / (DAC_MAX/2.0f)) -
13        1.0f;
14    float dac_float = (DAC_MAX/2.0f) + (DAC_MAX/2.0f) * amplitude *
15        sin_norm;
16    DAC_SetChannel1Data(DAC_Align_12b_R, (uint16_t)dac_float);
17
18    // Acquisition ADC retour DAC
19    float v_adc = ((float)readADC1(4) * VREF) / 4095.0f;
20    float v_ac = v_adc - (VREF / 2.0f);
21
22    // Calcul RMS
23    s += v_ac * v_ac;
24    rms_cnt++;
25    if (rms_cnt >= NPOINTS)
26    {
27        tension_eff = sqrtf(s / rms_cnt);
28        s = 0.0f;
29        rms_cnt = 0;
30    }
31 }
```

Explication : Cette routine assure le traitement temps réel avec un pas de temps constant, garantissant un signal stable et un calcul RMS précis. Toute la logique critique est placée dans l'interruption pour respecter les contraintes temporelles.

6 Résultats expérimentaux

Dans cette partie, nous présentons les résultats obtenus lors de l'acquisition et de la génération du signal sinusoïdal avec le STM32F407. Les mesures ont été effectuées à l'aide d'un oscilloscope et d'un analyseur logique.

6.1 Observation du signal

- Le signal sinusoïdal est généré par le DAC sur la broche PA4.
- Le retour du signal est acquis via l'ADC (PA4) et permet de calculer la valeur efficace RMS.
- Les figures 10 et 11 montrent respectivement l'oscillogramme et la capture sur le Logic Analyzer.

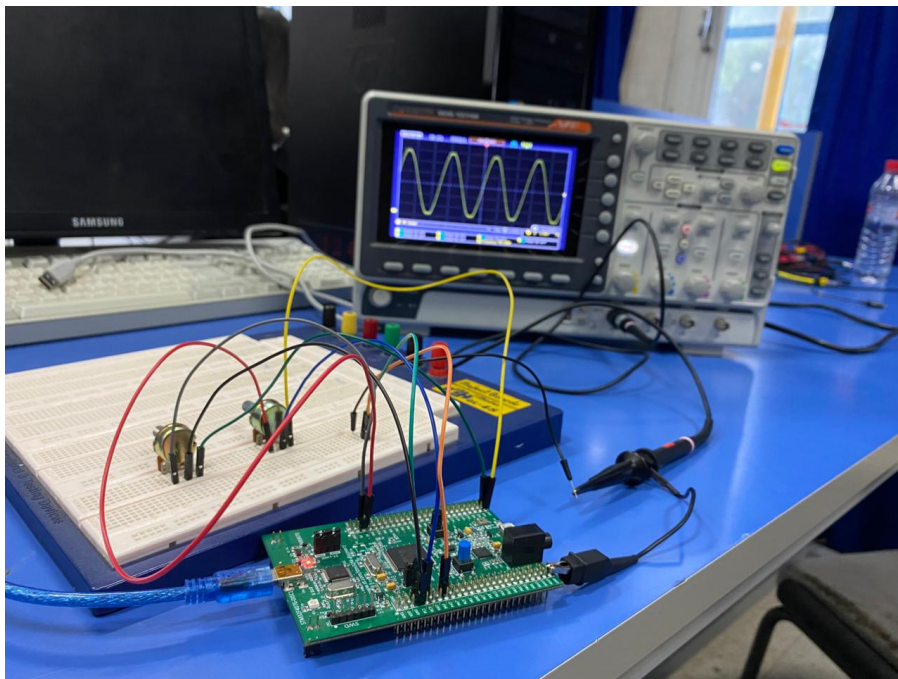


FIGURE 10 – Signal sinusoïdal généré et visualisé sur l'oscilloscope

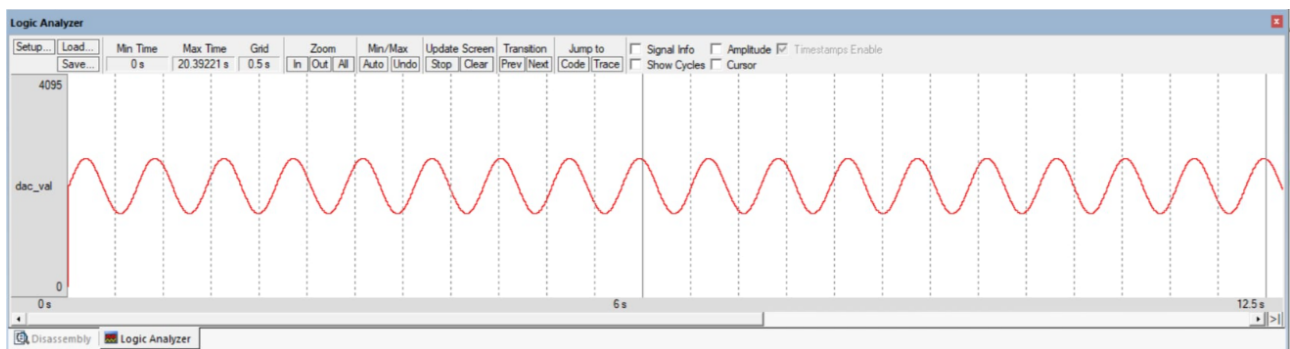


FIGURE 11 – Capture du signal sur le Logic Analyzer

6.2 Variation de la fréquence

Pour observer l'effet de la variation de la fréquence sur le signal généré, deux captures d'oscilloscope ont été réalisées avec des fréquences différentes, en maintenant l'amplitude constante.

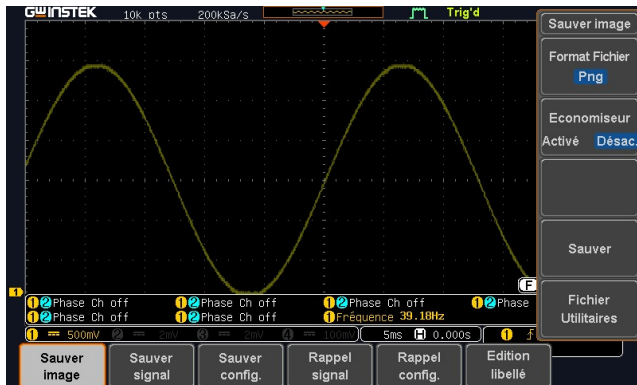


FIGURE 12 – Signal sinusoïdal à 39 Hz

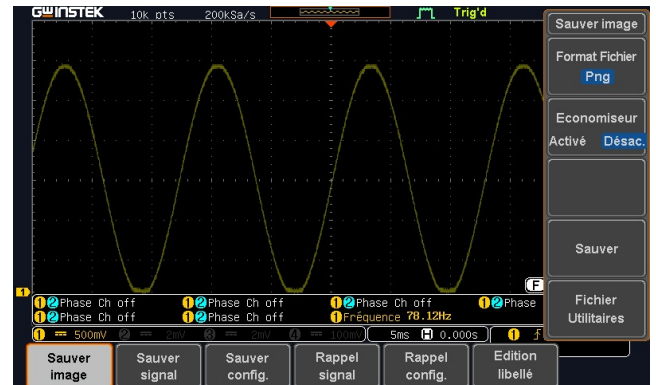


FIGURE 13 – Signal sinusoïdal à 78 Hz

La fréquence mesurée de 78,12 Hz est obtenue car le calcul du diviseur de fréquence conduit à $diviseur_freq = 1$, d'où la fréquence théorique :

$$f = \frac{F_s}{NPOINTS} = \frac{20000}{256} = 78,125 \text{ Hz},$$

ce qui est en accord avec la valeur observée à l'oscilloscope.

7 Conclusion

Le projet a consisté à générer un signal sinusoïdal avec un microcontrôleur STM32F407, à acquérir ce signal via un ADC et à calculer sa valeur efficace (RMS) en temps réel. L'ensemble du traitement a été piloté par une interruption SysTick assurant un échantillonnage stable et régulier.

Bilan du projet

- La génération du signal sinusoïdal via le DAC a été réalisée avec succès, avec une amplitude et une fréquence modulables à l'aide de potentiomètres.
- L'acquisition du signal par l'ADC a permis de mesurer la tension maximale et d'effectuer le calcul RMS avec précision.
- La routine d'interruption SysTick a garanti le traitement temps réel et la synchronisation correcte des conversions et de la génération du signal.
- Les résultats expérimentaux ont confirmé la conformité du système aux prévisions théoriques, notamment pour la relation $V_{\text{RMS}} = V_{\text{max}}/\sqrt{2}$ pour un sinus.

Objectifs atteints

- Compréhension et utilisation des périphériques DAC et ADC du STM32F407.
- Mise en œuvre d'une génération de signal sinusoïdal en temps réel.
- Acquisition et traitement du signal pour le calcul de la valeur efficace RMS.
- Visualisation des résultats à l'aide d'oscilloscope et Logic Analyzer.

Apports techniques

- Maîtrise des interruptions périodiques (SysTick) pour la gestion du temps réel.
- Développement modulaire en C pour un microcontrôleur ARM Cortex-M4.
- Gestion des conversions numériques-analogiques et analogiques-numériques avec précision.
- Analyse expérimentale et validation des résultats par des mesures directes.

Ce projet constitue une base solide pour des travaux futurs, tels que l'implémentation d'un calcul RMS plus précis sur plusieurs cycles, l'utilisation de DMA pour l'acquisition ADC, ou le développement d'une interface utilisateur pour la visualisation en temps réel.

A Annexes

A.1 Code source complet

Le code suivant présente l'intégralité des fichiers .c utilisés dans le projet : main.c, adc.c, dac.c et config.c. Chaque fichier est commenté pour clarifier son rôle.

Listing 6 – main.c – Initialisation, génération sinus et calcul RMS

```
1 #include "stm32f4xx.h"
2 #include "stm32f4xx_dac.h"
3 #include "stm32f4xx_adc.h"
4 #include "stm32f4xx_gpio.h"
5 #include "stm32f4xx_rcc.h"
6 #include <math.h>
7 #include <stdint.h>
8
9 #define PI2          6.283185307f
10 #define VREF         3.3f
11 #define FS           20000.0f
12 #define NPOINTS      256
13 #define DAC_MAX      4095.0f
14
15 uint16_t sin_table[NPOINTS];
16 volatile uint16_t dac_val, adc_val;
17 volatile uint16_t index_sin = 0;
18 volatile uint32_t freq_cnt = 0, diviseur_freq = 1;
19 volatile float s = 0.0f;           // somme des carr s pour RMS
20 volatile uint16_t rms_cnt = 0;
21 volatile float tension_eff = 0.0f;
22 volatile float dbg_sin_norm;
23
24 // Prototypes
25 void config(void);
26 void dac_init(void);
27 void adc_init(void);
28 uint16_t readADC1(uint8_t channel);
29 void init_sin_table(void);
30
31 // Initialisation table sinus
32 void init_sin_table(void)
33 {
34     for (int i = 0; i < NPOINTS; i++)
35         sin_table[i] = (uint16_t)((sinf(PI2 * i / NPOINTS) + 1.0f) * (
36             DAC_MAX / 2.0f));
37 }
38
39 int main(void)
40 {
41     config();           // activation horloges
42     dac_init();         // configuration DAC
43     adc_init();         // configuration ADC
44     init_sin_table();   // table sinus
45
46     SysTick_Config(SystemCoreClock / FS); // interruption SysTick
```

```

47     while (1)
48     {
49         // traitement temps r el g r dans SysTick_Handler
50     }
51 }
52
53 // Routine d'interruption SysTick
54 void SysTick_Handler(void)
55 {
56     uint16_t pot_amp = readADC1(11); // amplitude
57     uint16_t pot_freq = readADC1(12); // fr quence
58     float amplitude = (float)pot_amp / 4095.0f;
59     float freq = 1.0f + ((float)pot_freq / 4095.0f) * 99.0f;
60
61     diviseur_freq = (uint32_t)(FS / (freq * NPOINTS));
62     if (diviseur_freq < 1) diviseur_freq = 1;
63
64     freq_cnt++;
65     if (freq_cnt >= diviseur_freq)
66     {
67         freq_cnt = 0;
68         float sin_norm = ((float)sin_table[index_sin] / (DAC_MAX/2.0f))
69             - 1.0f;
70         dbg_sin_norm = sin_norm;
71         float dac_float = (DAC_MAX/2.0f) + (DAC_MAX/2.0f) * amplitude *
72             sin_norm;
73         if(dac_float < 0.0f) dac_float = 0.0f;
74         if(dac_float > DAC_MAX) dac_float = DAC_MAX;
75         dac_val = (uint16_t)dac_float;
76         DAC_SetChannel1Data(DAC_Align_12b_R, dac_val);
77         index_sin++;
78         if(index_sin >= NPOINTS) index_sin = 0;
79     }
80
81     adc_val = readADC1(4); // retour DAC
82     float v_adc = ((float)adc_val * VREF) / 4095.0f;
83     float v_ac = v_adc - (VREF / 2.0f);
84     s += v_ac * v_ac;
85     rms_cnt++;
86     if(rms_cnt >= NPOINTS)
87     {
88         tension_eff = sqrtf(s / rms_cnt);
89         s = 0.0f;
90         rms_cnt = 0;
91     }
92 }

```

Listing 7 – adc.c – Initialisation et lecture ADC

```

1 #include "stm32f4xx.h"
2 #include "stm32f4xx_adc.h"
3 #include "stm32f4xx_gpio.h"
4
5 void adc_init(void)
6 {

```

```

7   ADC_InitTypeDef ADC_InitStruct;
8   ADC_CommonInitTypeDef ADC_CommonInitStruct;
9   GPIO_InitTypeDef GPIO_InitStruct;
10
11  // PC0, PC1, PC2 analogiques
12  GPIO_InitStruct.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2;
13  GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AN;
14  GPIO_InitStruct.GPIO_PuPd = GPIO_PuPd_NOPULL;
15  GPIO_Init(GPIOC, &GPIO_InitStruct);
16
17  // PA4 analogique
18  GPIO_InitStruct.GPIO_Pin = GPIO_Pin_4;
19  GPIO_Init(GPIOA, &GPIO_InitStruct);
20
21  ADC_CommonInitStruct.ADC_Mode = ADC_Mode_Independent;
22  ADC_CommonInitStruct.ADC_Prescaler = ADC_Prescaler_Div4;
23  ADC_CommonInitStruct.ADC_DMAAccessMode = ADC_DMAAccessMode_Disabled
24  ;
25  ADC_CommonInitStruct.ADC_TwoSamplingDelay =
26      ADC_TwoSamplingDelay_5Cycles;
27  ADC_CommonInit(&ADC_CommonInitStruct);
28
29  ADC_InitStruct.ADC_Resolution = ADC_Resolution_12b;
30  ADC_InitStruct.ADC_ScanConvMode = DISABLE;
31  ADC_InitStruct.ADC_ContinuousConvMode = DISABLE;
32  ADC_InitStruct.ADC_ExternalTrigConvEdge =
33      ADC_ExternalTrigConvEdge_None;
34  ADC_InitStruct.ADC_DataAlign = ADC_DataAlign_Right;
35  ADC_InitStruct.ADC_NbrOfConversion = 1;
36  ADC_Init(ADC1, &ADC_InitStruct);
37  ADC_Cmd(ADC1, ENABLE);
38 }
39
40 uint16_t readADC1(uint8_t channel)
41 {
42     ADC-RegularChannelConfig(ADC1, channel, 1, ADC_SampleTime_3Cycles);
43     ADC_SoftwareStartConv(ADC1);
44     while (!ADC_GetFlagStatus(ADC1, ADC_FLAG_EOC));
45     return ADC_GetConversionValue(ADC1);
46 }

```

Listing 8 – dac.c – Initialisation DAC

```

1  #include "stm32f4xx.h"
2  #include "stm32f4xx_dac.h"
3  #include "stm32f4xx_gpio.h"
4
5  void dac_init(void)
6  {
7      DAC_InitTypeDef DAC_InitStruct;
8      GPIO_InitTypeDef GPIO_InitStruct;
9
10     // DAC1 sur PA4
11     GPIO_InitStruct.GPIO_Pin = GPIO_Pin_4;
12     GPIO_InitStruct.GPIO_Mode = GPIO_Mode_AN;

```



```

13  GPIO_InitStruct.GPIO_PuPd = GPIO_PuPd_NOPULL;
14  GPIO_Init(GPIOA, &GPIO_InitStruct);
15
16  DAC_InitStruct.DAC_Trigger = DAC_Trigger_None;
17  DAC_InitStruct.DAC_OutputBuffer = DAC_OutputBuffer_Enable;
18  DAC_Init(DAC_Channel_1, &DAC_InitStruct);
19
20  DAC_Cmd(DAC_Channel_1, ENABLE);
21 }

```

Listing 9 – config.c – Activation horloges et GPIO

```

1  #include "stm32f4xx.h"
2  #include "stm32f4xx_rcc.h"
3
4  void config(void)
5  {
6      RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA | RCC_AHB1Periph_GPIOC,
7                              ENABLE);
8      RCC_APB1PeriphClockCmd(RCC_APB1Periph_DAC, ENABLE);
9      RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC1, ENABLE);
10 }

```

B Montage complet du système

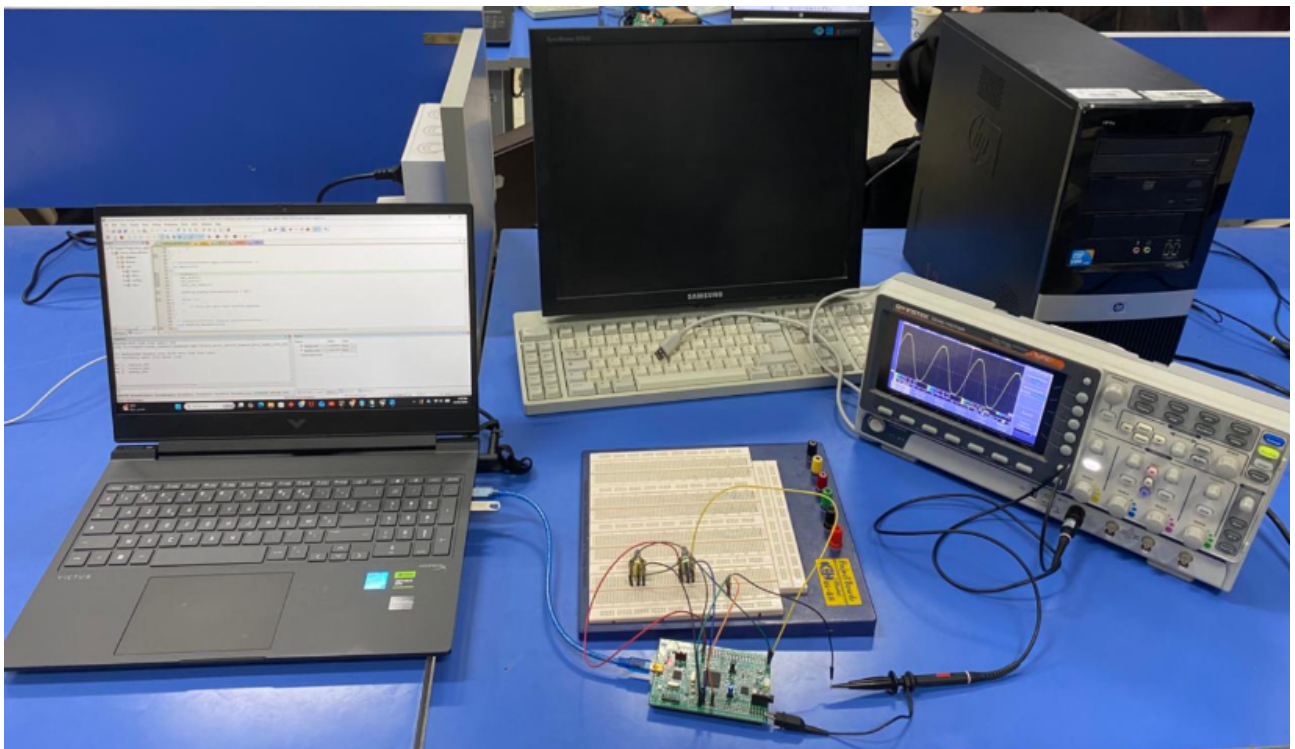


FIGURE 14 – Montage complet du système